Arquitetura do projeto

O que é o projeto

O projeto é um sistema de **monitoramento legislativo** com foco na proteção de crianças e adolescentes na internet. O objetivo é acompanhar automaticamente os projetos de lei e proposições que tramitam na Câmara dos Deputados e no Senado Federal, identificando quais deles estão relacionados a temas como cyberbullying, privacidade de dados de menores, tempo de tela, conteúdo inapropriado e educação digital.

Em vez de alguém precisar acessar manualmente os sites do governo e procurar esses projetos, o sistema faz isso de forma automática: coleta os dados, filtra o que é relevante, classifica por tema usando inteligência artificial, armazena tudo organizado e exibe as informações em um portal visual acessível a qualquer pessoa.

Por que Pipes and Filters

Existem vários estilos de arquitetura de software — monolítica, microsserviços, MVC, orientada a eventos, entre outros. Para este projeto, a escolha foi a arquitetura **Pipes and Filters**, e a razão é direta: o sistema funciona naturalmente como uma linha de processamento de dados.

Pipes and Filters é um estilo arquitetural onde os dados passam por uma sequência de etapas independentes. Cada etapa (filter) faz uma transformação específica nos dados e passa o resultado para a próxima através de um canal (pipe). Nenhuma etapa precisa saber como as outras

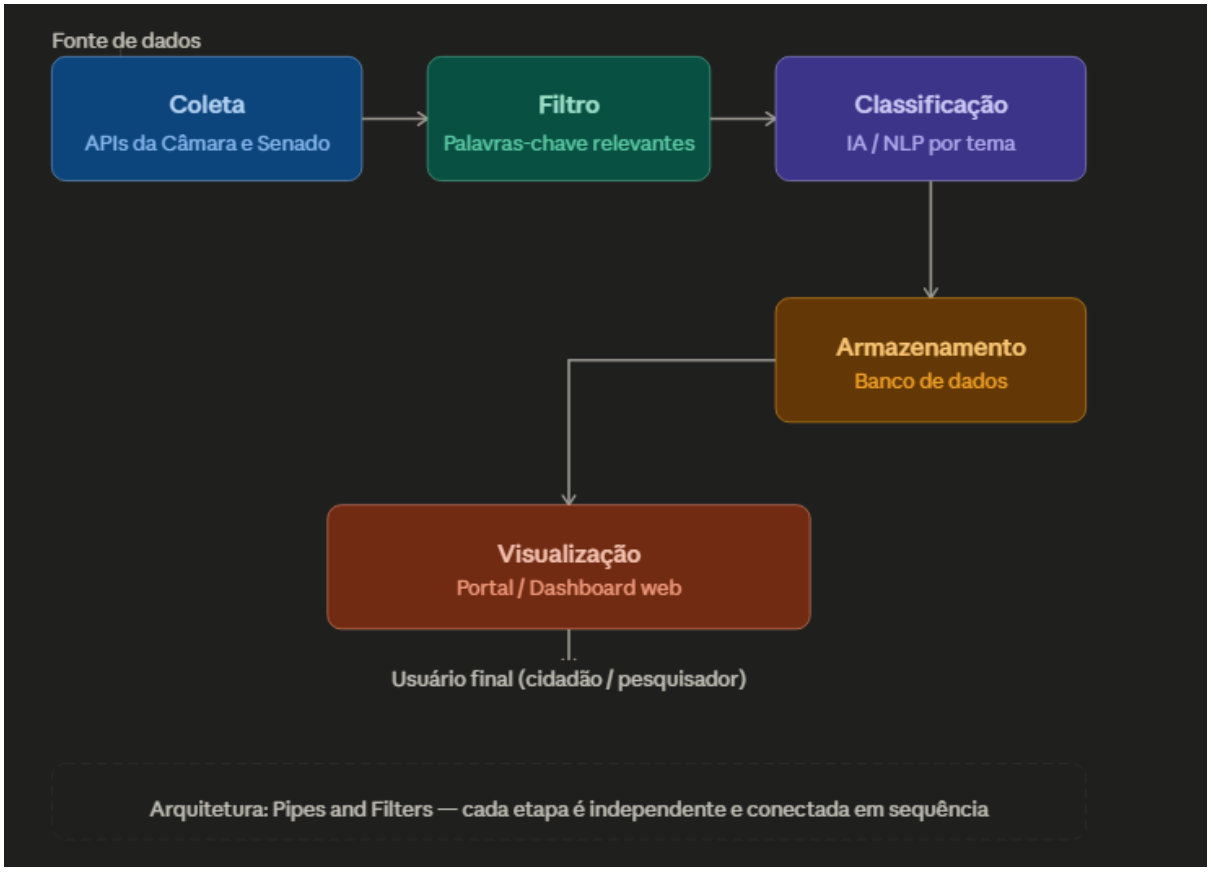
funcionam — cada uma recebe uma entrada, processa, e entrega uma saída.

Esse estilo se encaixa perfeitamente com o fluxo do projeto:

- Os dados brutos são **coletados** das APIs públicas
- Passam por um **filtro** de palavras-chave
- São **classificados** por tema via IA
- São **armazenados** no banco de dados
- São **exibidos** no portal

Cada etapa é independente, pode ser desenvolvida separadamente por membros diferentes da equipe, e pode ser modificada ou substituída sem quebrar as demais. Isso facilita o trabalho em equipe, a manutenção e a evolução do sistema ao longo das sprints.

Esboço do Projeto



Me explica melhor a etapa de Coleta de dados do projeto de monitoramento legislativo

A etapa de Coleta é a **entrada do pipeline** — sem ela, o sistema não tem nada pra processar.

O que acontece nessa etapa

O sistema vai nas APIs públicas da Câmara e do Senado e baixa automaticamente os dados de proposições legislativas (projetos de lei, emendas, requerimentos, etc.). Isso é feito por um script Python que roda periodicamente — tipo, todo dia de madrugada — e busca o que é novo.

As APIs que você vai usar

As duas são públicas e gratuitas, sem precisar de login:

- **Câmara:** https://dadosabertos.camara.leg.br/api/v2/
- **Senado:** https://legis.senado.leg.br/dadosabertos/

Ambas retornam dados em JSON, e têm endpoints específicos pra buscar proposições por data, tema, autor, etc.

O que o script faz na prática

```python
import requests

# Exemplo: buscar proposições da Câmara de hoje
response = requests.get(
    "https://dadosabertos.camara.leg.br/api/v2/proposicoes",
    params={
        "dataInicio": "2025-01-01",
        "dataFim": "2025-01-31",
        "itens": 100
    }
)

dados = response.json()
```

Você chama a API, pega o JSON de resposta, e salva em algum lugar (arquivo, banco de dados) pra passar pra próxima etapa.

O que é salvo (dados brutos)

Cada proposição tem campos como título, ementa (resumo), autor, data, partido, estado do parlamentar, situação atual, etc. Tudo isso vai ser útil nas etapas seguintes.

Na sua arquitetura, a responsabilidade da etapa de Coleta é **só buscar e salvar** — ela não filtra, não classifica, não analisa nada. Isso é intencional no Pipes and Filters: cada etapa faz uma coisa só.

Me explica melhor a etapa de filtro por palavras-chave do projeto

A etapa de Filtro é a **segunda do pipeline** — ela pega tudo que foi coletado e descarta o que não interessa.

O problema que ela resolve

A API da Câmara tem milhares de proposições. A grande maioria não tem nada a ver com crianças ou internet — fala de agropecuária, impostos, obras públicas, etc. Sem um filtro, a IA da etapa seguinte teria que processar tudo isso à toa, gastando tempo e recursos.

Como funciona na prática

O script lê os dados brutos salvos pela etapa de Coleta e verifica se o texto de cada proposição contém pelo menos uma das palavras-chave do seu tema:

```python
palavras_chave = [
    "criança", "adolescente", "menor",
    "internet", "digital", "online",
    "cyberbullying", "cybercrime",
    "Marco Civil", "LGPD",
    "redes sociais", "aplicativo",
    "proteção de dados", "ECA"
]

def filtrar(proposicao):
    texto = proposicao["ementa"].lower()
    return any(palavra.lower() in texto for palavra in palavras_chave)

proposicoes_filtradas = [p for p in proposicoes_brutas if filtrar(p)]
```

Simples assim — se a ementa contém qualquer uma dessas palavras, a proposição passa. Se não contém, é descartada.

O que entra e o que sai

	Quantidade estimada
Proposições brutas (entrada)	~5.000 por mês
Proposições filtradas (saída)	~50–200 por mês

Você reduz drasticamente o volume antes de chamar a IA, que é a parte mais cara computacionalmente.

Um detalhe importante de arquitetura

O filtro por palavras-chave é **intencional mente simples** — ele vai deixar passar alguns falsos positivos (proposições que citam "criança" mas falam de outra coisa) e pode perder alguns casos mais sutis. Tudo bem, porque a etapa de Classificação com IA logo depois vai refinar isso com muito mais inteligência.

A divisão de responsabilidades fica assim:

- **Filtro** → corte rápido e barato por palavras
- **Classificação** → análise profunda e precisa por IA

Me explica melhor a etapa de Classificação com IA/NLP do projeto

A etapa de Classificação é o **coração inteligente do pipeline** — é aqui que o sistema entende o que cada proposição realmente trata.

O problema que ela resolve

Depois do filtro, você tem proposições que *provavelmente* falam sobre crianças e internet. Mas você não sabe **qual aspecto** cada uma aborda. Uma fala de cyberbullying, outra de tempo de tela, outra de pornografia infantil, outra de privacidade de dados. A classificação organiza isso.

O que é NLP

NLP (Natural Language Processing) é a área da IA que trabalha com texto. No seu caso, você vai usar modelos já prontos — não precisa treinar nada do zero. A biblioteca mais usada em Python pra isso é a **spaCy** ou **transformers** da Hugging Face.

Como funciona na prática

A abordagem mais simples pro seu projeto acadêmico é usar um modelo pré-treinado em português pra calcular a similaridade entre o texto da proposição e categorias pré-definidas:

```python
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

modelo = SentenceTransformer('neuralmind/bert-base-portuguese-cased')

# Categorias que você quer identificar
categorias = {
    "cyberbullying":        "assédio, bullying, violência psicológica online",
    "privacidade de dados": "coleta de dados pessoais, LGPD, proteção de informações
    "tempo de tela":        "uso excessivo, dependência digital, limite de uso",
    "conteúdo inapropriado":"pornografia, violência, conteúdo adulto na internet",
    "educação digital":     "letramento digital, ensino, alfabetização tecnológica"
}

def classificar(texto_proposicao):
    emb_texto = modelo.encode([texto_proposicao])
    emb_cats  = modelo.encode(list(categorias.values()))

    similaridades = cosine_similarity(emb_texto, emb_cats)[0]
    idx_melhor    = np.argmax(similaridades)

    return list(categorias.keys())[idx_melhor]
```

O modelo transforma o texto em números (vetores), e aí compara matematicamente qual categoria é mais parecida com o texto da proposição.

O que sai dessa etapa

Cada proposição ganha uma **tag de categoria** e uma **pontuação de confiança**:

```json
{
  "id": "PL-1234/2024",
  "ementa": "Dispõe sobre limites de uso de redes sociais por menores...",
  "categoria": "tempo de tela",
  "confianca": 0.87
}
```

Esses dados enriquecidos aí passam pro Armazenamento e depois aparecem organizados no Dashboard.

Me explica melhor a etapa de Armazenamento de dados do projeto

A etapa de Armazenamento é a **memória do sistema** — é onde tudo que foi processado fica guardado de forma organizada pra ser consultado pelo Dashboard.

O problema que ela resolve

Sem armazenamento estruturado, cada vez que o usuário abrisse o Dashboard o sistema teria que coletar, filtrar e classificar tudo do zero — o que seria lento e inviável. O banco de dados guarda o resultado final do pipeline pra consulta rápida.

Qual banco usar

Pro seu projeto acadêmico, o mais indicado é o **PostgreSQL** — é gratuito, robusto, e vai bem com Python e Docker (que já estão no escopo do projeto).

Como os dados ficam organizados

A estrutura principal seria uma tabela assim:

```sql
CREATE TABLE proposicoes (
    id                SERIAL PRIMARY KEY,
    id_externo        VARCHAR(50),        -- ex: "PL-1234/2024"
    titulo            TEXT,
    ementa            TEXT,
    autor             VARCHAR(200),
    partido           VARCHAR(20),
    estado            VARCHAR(2),
    casa              VARCHAR(10),        -- "camara" ou "senado"
    data_apresentacao DATE,
    categoria         VARCHAR(100),       -- resultado da classificação
    confianca         FLOAT,              -- pontuação da IA
    coletado_em       TIMESTAMP DEFAULT NOW()
);
```

Cada linha é uma proposição que passou pelo pipeline inteiro — coletada, filtrada e classificada.

Como o Python salva os dados

```python
import psycopg2

def salvar(proposicao):
    conn = psycopg2.connect("postgresql://usuario:senha@localhost/legislativo")
    cur  = conn.cursor()

    cur.execute("""
        INSERT INTO proposicoes
            (id_externo, titulo, ementa, autor, partido,
             estado, casa, data_apresentacao, categoria, confianca)
        VALUES
            (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """, (
        proposicao["id"],
        proposicao["titulo"],
        proposicao["ementa"],
        proposicao["autor"],
        proposicao["partido"],
        proposicao["estado"],
        proposicao["casa"],
        proposicao["data"],
        proposicao["categoria"],
        proposicao["confianca"]
    ))

    conn.commit()
```

O que o Dashboard vai fazer com isso

O Dashboard simplesmente faz consultas SQL nessa tabela pra montar os gráficos:

Exatamente os indicadores que o projeto pede mostrar.

```sql
-- Quantidade por categoria
SELECT categoria, COUNT(*) FROM proposicoes GROUP BY categoria;

-- Parlamentares mais ativos
SELECT autor, COUNT(*) FROM proposicoes GROUP BY autor ORDER BY COUNT(*) DESC;

-- Evolução ao longo do tempo
SELECT DATE_TRUNC('month', data_apresentacao), COUNT(*)
FROM proposicoes GROUP BY 1 ORDER BY 1;
```

Me explica melhor a etapa de Visualização Dashboard do projeto

A etapa de Visualização é a **face do sistema** — é a única parte que o usuário final vai ver e interagir. Tudo que foi feito antes existe pra chegar aqui.

O que o Dashboard precisa mostrar

O projeto já define isso claramente:

- Quantidade de proposições por tema/categoria
- Evolução ao longo do tempo (gráfico de linha)
- Parlamentares mais ativos (ranking)
- Distribuição por partido e região
- Identificação de novos temas

Qual tecnologia usar

Como o projeto já usa Python no backend, a escolha mais natural é o **Streamlit** — você escreve o Dashboard inteiro em Python, sem precisar de HTML/CSS/JavaScript. É a ferramenta padrão pra projetos acadêmicos de dados.

A alternativa seria React ou Vue no frontend, mas isso adiciona complexidade desnecessária pro escopo do projeto.

Como funciona na prática

python

```python
import streamlit as st
import pandas as pd
import psycopg2
import plotly.express as px

# Conecta no banco
@st.cache_data
def carregar_dados():
    conn = psycopg2.connect("postgresql://usuario:senha@localhost/legislativo")
    return pd.read_sql("SELECT * FROM proposicoes", conn)

df = carregar_dados()

st.title("Monitoramento Legislativo — Proteção de Crianças na Internet")

# Gráfico 1: proposições por categoria
fig1 = px.bar(
    df.groupby("categoria").size().reset_index(name="total"),
    x="categoria", y="total",
    title="Proposições por tema"
)
st.plotly_chart(fig1)

# Gráfico 2: evolução no tempo
fig2 = px.line(
    df.groupby("data_apresentacao").size().reset_index(name="total"),
    x="data_apresentacao", y="total",
    title="Evolução ao longo do tempo"
```


```python
    title="Evolução ao longo do tempo"
```


```python
)
st.plotly_chart(fig2)

# Tabela: parlamentares mais ativos
st.subheader("Parlamentares mais ativos")
st.dataframe(
    df.groupby("autor").size()
      .reset_index(name="proposições")
      .sort_values("proposições", ascending=False)
      .head(10)
)
```

Visão completa do pipeline agora

Você agora conhece todas as 5 etapas. Resumindo as responsabilidades de cada uma:

Etapa	Tecnologia	Responsável no grupo
Coleta	Python + requests	Dados
Filtro	Python puro	Dados
Classificação	Python + transformers	Dados / IA
Armazenamento	PostgreSQL	Backend
Visualização	Streamlit + Plotly	Frontend